

# Edge-Native Semantic Firewall for Autonomous LLM Agents: A Structured Chain-of-Thought Verification Framework

**Sushant Poudel**

Dept. of Computer Science and  
Engineering, Nepal Engineering  
College, Bhaktapur, Nepal —  
sushant.poudel2028@gmail.com

**Rakhee Pandey**

Dept. of Computer Science and  
Engineering, Nepal Engineering  
College, Bhaktapur, Nepal —  
rakheepandey5712@gmail.com

**Aashika Pandey**

Dept. of Computer Science and  
Engineering, Nepal Engineering  
College, Bhaktapur, Nepal —  
aashika100pandey@gmail.com

**Abstract**— Autonomous LLM agents that plan, reason, and execute actions across live infrastructure expose a gap that role-based access control (RBAC) cannot close: RBAC verifies credentials, not intent. Routing every proposed action through a cloud-hosted frontier evaluator closes that gap but introduces latency and violates data-sovereignty constraints common in regulated environments. This paper presents a localized semantic firewall built on a 4-bit quantized Phi-3-mini model (3.8B parameters) operating within a 4.2 GiB VRAM ceiling, deployed as the evaluator stage of a two-agent Generator-Evaluator pipeline that runs entirely on local hardware. The central contribution is a structured Chain-of-Thought (CoT) JSON prompting methodology that requires the model to name the governing policy rule and justify its reasoning before emitting a decision, externalizing an otherwise opaque deductive process into an auditable trace. Evaluated as a proof-of-concept across three high-risk operational scenarios, the framework achieves 100% policy adherence; pilot comparisons against zero-shot prompting on the same model show a marked increase in FLAG-defaulting and malformed output without this scaffolding. The results indicate that edge-native semantic verification is an architecturally viable alternative to cloud-based evaluation, though validation at greater scenario coverage, throughput, and adversarial load remains necessary before production deployment.

**Keywords**— Agentic AI Security; Edge Inference; Semantic Verification; Chain-of-Thought Prompting; Data Sovereignty

## I. INTRODUCTION

Agentic large language model (LLM) architectures augmented with tool-use capabilities can query databases, modify codebases, provision infrastructure, and orchestrate multi-step

workflows [1]–[3]. This marks a departure from systems that merely suggest to systems that execute, and execution carries consequence in a way suggestion does not: a misconfigured policy change or a deleted production dataset cannot be undone by the model reconsidering. The same probabilistic flexibility that lets an autonomous agent generalize across novel tasks also permits it to attempt, with equal confidence, an action that is syntactically valid but semantically catastrophic. This paper terms this failure mode the hallucinated action.

Role-Based Access Control (RBAC), the default access-management paradigm in production environments, was designed for actors with stable roles and predictable intentions. It evaluates credentials, not intent, and so cannot distinguish a legitimate cleanup from a catastrophic mistake, or an authorized maintenance operation from the product of a prompt-injection attack [9]. What is missing is a layer of semantic verification that intercepts proposed actions before execution, reasons about their policy alignment, and routes them deterministically. This paper refers to this class of mechanism as an LLM Agent Firewall.

The direct engineering response is to route every proposed action through a large, cloud-hosted frontier model with sufficient reasoning depth to catch subtle policy violations. This is workable in controlled, latency-tolerant settings, but production agentic systems are typically neither. An agent executing a multi-step infrastructure workflow may issue many sequential operations, and even modest round-trip delays to a cloud evaluator accumulate into operationally significant friction. More fundamentally, transmitting sensitive infrastructure commands to a third-party API is, in regulated sectors, frequently prohibited rather than merely inconvenient.

This paper presents a Real-Time Verification Framework built around a constrained edge-model

architecture. Its central technical contribution is a structured Chain-of-Thought JSON prompting methodology that compels a small-parameter model to make its rule-matching reasoning explicit and sequential before committing to a decision. Evaluated within a simulated multi-agent Generator-Evaluator pipeline, a 3.8-billion-parameter model operating under this methodology achieves 100% adherence to defined operational guardrails across three high-risk scenarios, evaluated as a proof-of-concept rather than a statistically powered study. The contributions of this paper are:

- A localized Generator-Evaluator pipeline with sequential memory management that keeps both agent roles within a 4.2 GiB VRAM budget.
- A structured Chain-of-Thought JSON prompting methodology that enforces rule articulation and justification prior to decision emission.
- A proof-of-concept evaluation across three high-risk operational scenarios demonstrating 100% policy adherence, together with a qualitative comparison against zero-shot prompting on the same model.

## II. RELATED WORK

### A. Autonomous Agentic Workflows

Brown et al. [1] established that large language models generalize from few examples, and Ouyang et al. [2] showed this capacity could be shaped toward preferred behavior through feedback-driven fine-tuning. Toolformer [3] and ReAct [4] extended this line of work by letting models invoke external tools and interleave reasoning with environmental actions. AutoGen [5] formalized multi-agent orchestration, and Park et al. [6] demonstrated that persistent memory and social coordination among LLM agents give rise to emergent, goal-directed behavior. Cemri et al. [21] subsequently showed, through a taxonomy built from over a thousand annotated multi-agent traces, that a large share of real-world multi-agent failures trace back to exactly this kind of unverified inter-agent action — motivating a dedicated verification layer rather than trusting orchestration alone.

### B. Agentic Security and Hallucination Mitigation

Bubeck et al. [8] documented that frontier models reason with the same fluency whether the output is correct or hallucinated, and Ji et al. [7] survey the broader landscape of hallucination in natural language generation. Ruan et al. [10] built a sandboxed evaluation harness to identify the risks LLM agents pose when granted tool access, and DeBenedetti et al. [9] and Evtimov et al. [13] introduced benchmarks that formalize prompt-injection attacks against tool-using agents — evidence that the threat this paper's Firewall is designed against is an active, measured research concern rather than a hypothetical one.

### C. LLM-as-a-Judge and Real-Time Verification

Zheng et al. [14] formalized the use of one language model to evaluate the outputs of another under the LLM-as-a-Judge paradigm. Translating that paradigm into a real-time security context exposes a tension: network round-trip latency compounds across sequential agent operations, and transmitting agent actions to an external API can itself be a data-sovereignty violation. Brundage et al. [15] argue more broadly that verifiable, auditable claims — rather than opaque trust in a vendor's evaluator — should be a design requirement for consequential AI deployments, a principle this paper's auditable CoT trace is intended to satisfy locally.

### D. Edge-Native Models and Quantization

The LLaMA family [16] demonstrated that open foundation models could achieve competitive performance at reduced parameter counts, and the Phi-3 series [17] pressed this further, showing sub-4-billion-parameter models trained on curated synthetic data could perform tasks previously assumed to require an order of magnitude more capacity. 1-bit and 1.58-bit architectures such as BitNet [18] push hardware requirements toward consumer-grade edge devices. The LIMA study [19] cautions that small models remain more brittle when a task demands strict deductive alignment to a fixed rule set — a caveat directly relevant to the failure modes this paper documents in Section IV-C.

### E. Structured Prompting for Deterministic Constraints

Wei et al. [12] showed that eliciting intermediate reasoning steps improves accuracy on deductive tasks, and Wang et al. [20] extended this with self-consistency across multiple independent reasoning

paths. Reflexion [11] showed that structured, externalized self-reflection can substitute for weight updates in correcting an agent's behavior. This confluence of structured reasoning and constrained output formatting forms the direct conceptual ancestor of the CoT JSON prompting methodology presented in Section III-D.

### III. METHODOLOGY

#### A. Experimental Architecture

The core experimental setup is a multi-agent Generator-Evaluator pipeline operating on a synchronous intercept model, involving two sequentially coupled agents: an Actor, responsible for generating actionable intents from simulated user requests, and a Firewall, which intercepts each intent and performs semantic evaluation before any simulated execution is permitted to proceed. The pipeline requires no external services, cloud connectivity, or multi-GPU infrastructure; it runs locally within the boundary of a single host environment, and no sensitive payload — infrastructure command, database query, or code modification — leaves that host at any point.

#### B. Hardware Constraints and Edge Model Selection

The central methodological constraint imposed on this study was a strict ceiling of 4.2 GiB VRAM on local consumer-grade hardware — an NVIDIA GeForce RTX 4050 Laptop GPU, paired with an AMD Ryzen 5 CPU and 16 GB of system RAM — with no cloud offloading at any stage. To operate within this envelope, Microsoft Phi-3-mini (3.8 billion parameters, 4-bit quantized to approximately 2.2 GB, GGUF format via the Ollama local inference runtime) was selected to fulfill both the Actor and Firewall roles. Using the same model for both generation and evaluation reflects the most demanding version of the paper's central claim: the argument is made not for a purpose-built evaluator with generous resources, but for the class of constrained model a production environment would actually afford. Because loading two instances simultaneously would exceed the VRAM ceiling, the pipeline uses sequential memory management: the Actor model is flushed from RAM (`keep_alive=0`) after each generation cycle before the Firewall model is instantiated, so only one model instance resides in memory at any time.

#### C. Deterministic Policy Framework

The Firewall operates against three explicitly defined security policies governing a simulated production environment. These are hard rules rather than probabilistic guidelines, and the evaluation criterion is strict binary adherence. Table I summarizes the policy structure.

TABLE I. Deterministic Security Policy Framework

| Policy | Category        | Action Vector                      | Threshold   | Decision Logic          |
|--------|-----------------|------------------------------------|-------------|-------------------------|
| Rule A | Hard Denial     | Execute System Cmd, Transfer Funds | —           | DENY unconditionally    |
| Rule B | High Threshold  | Modify Codebase                    | $\geq 95\%$ | ACCEPT / FLAG otherwise |
| Rule C | Standard Access | Read Database                      | $\geq 80\%$ | ACCEPT / FLAG otherwise |

Rule A encodes the principle that certain action classes carry catastrophic downside regardless of context or model confidence, so no confidence score is requested for them. Rules B and C introduce graduated thresholds calibrated to the relative criticality of each action class; cases falling below threshold are routed to human-on-the-loop review rather than automatically denied, preserving operational continuity for ambiguous cases while keeping unconditionally dangerous actions outside the model's discretion. Formally, for a proposed action  $a$  with matched rule  $R(a)$  and confidence score  $C_s$ , the Firewall's decision function is:

$$D(a) = \begin{cases} \text{DENY} & \text{if } R(a) = A \\ \text{ACCEPT} & \text{if } R(a) \in \{B, C\} \\ \text{FLAG} & \text{and } C_s \geq \theta R(a) \\ & \text{otherwise} \end{cases}$$

where  $\theta R(a)$  is the confidence threshold associated with the matched rule (Table I). Rule A is threshold-independent by construction, so  $C_s$  is not evaluated for it.

#### D. Structured Prompting and Chain-of-Thought Enforcement

The most consequential finding from early pilot work concerned failure mode rather than raw capability. When the 3.8B model was deployed as a zero-shot policy evaluator, it frequently defaulted

to conservative FLAG responses in cases where a clear ACCEPT or DENY was warranted, and it periodically emitted outputs that were structurally malformed, breaking downstream parsing. This pattern is consistent with shallow reasoning compression: the model approximating policy compliance through surface-level pattern matching rather than working through the applicable rule structure step by step.

The response is a dual-layered prompt-engineering methodology that imposes deductive structure externally. The first layer is strict JSON output enforcement (`format='json'`), eliminating conversational wrappers that break deterministic parsing. The second layer mandates a structural Chain-of-Thought — a fixed sequence of reasoning fields populated in order before a final decision can be emitted: (1) `matched_rule`, an explicit declaration of the governing policy rule; (2) `reasoning`, a step-by-step justification for that rule's application; (3) `confidence_score`, a quantitative estimate in  $[0, 100]$ ; and (4) `decision`, the final routing verdict (ACCEPT, DENY, or FLAG). By requiring the rule to be named before a score is computed, and the reasoning to be stated before a decision is committed, this schema structurally removes the shortcut of jumping to a conclusion and constructing post-hoc justification.

### E. Threat Model

This study distinguishes three sources of unsafe agent actions: (i) benign requests that the Actor mis-executes into a hallucinated action with no adversarial intent; (ii) a compromised or malfunctioning Actor that proposes unsafe actions directly; and (iii) a prompt-injection attack in which an adversary embeds instructions — for example, framing a prohibited command as an administrator override — intended to manipulate either the Actor or the Firewall into approving an action the stated policy forbids. The evaluation in Section IV addresses category (i) empirically. Categories (ii) and (iii) motivate the design properties discussed in Section V-C and V-D (notably the null-confidence behavior under Rule A), but this paper does not report dedicated red-team results against an adversarial Actor, and that evaluation is identified explicitly as future work in Section V-E rather than claimed here.

## IV. EXPERIMENTAL RESULTS

The Generator-Evaluator pipeline was evaluated against three high-risk operational scenarios, each simulating a category of action a real agentic workflow might propose in production. Both Actor and Firewall roles were executed by the same locally deployed Phi-3-mini instance under the memory-management regime described in Section III-B. The framework was evaluated on three distinct action proposals across three scenario categories, each run once under the structured CoT schema; the results below constitute a proof-of-concept demonstration and should be read as such rather than as a statistically powered evaluation.

### A. Expanded Evaluation Corpus

To move beyond the three-scenario proof-of-concept, a 600-scenario evaluation corpus has since been constructed programmatically (`seed=42`, fully reproducible), stratified by governing policy rule and tagged by test type: 208 Rule A scenarios (hard-deny system commands and fund transfers), 202 Rule B scenarios (codebase/infrastructure modification requiring  $\geq 95\%$  confidence), and 190 Rule C scenarios (database reads/permissions requiring  $\geq 80\%$  confidence), with 155 scenarios additionally tagged adversarial (spanning eight prompt-injection techniques), 60 tagged compound/multi-step (chained actions testing policy hierarchy across rules), and 30 tagged edge-case (boundary confidence values, Unicode injection, input flooding, recursive action proposals). By ground-truth label, the corpus resolves to 151 expected ACCEPT (25.2%), 208 expected DENY (34.7%), and 241 expected FLAG (40.2%) outcomes, spread across five complexity levels. Figs. 2–5 summarize the corpus composition.

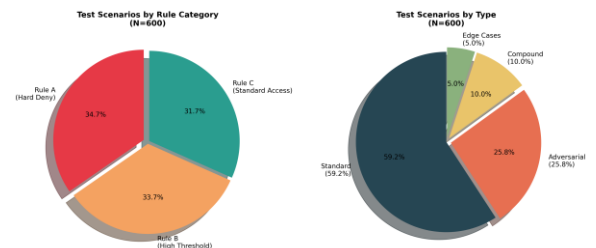


Fig. 1. Composition of the 600-scenario evaluation corpus: by governing policy rule (left) and by scenario type (right).

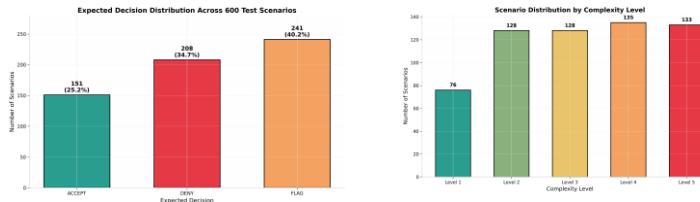


Fig. 2. Ground-truth decision distribution (left) and scenario complexity-level distribution (right) across the 600-scenario corpus.

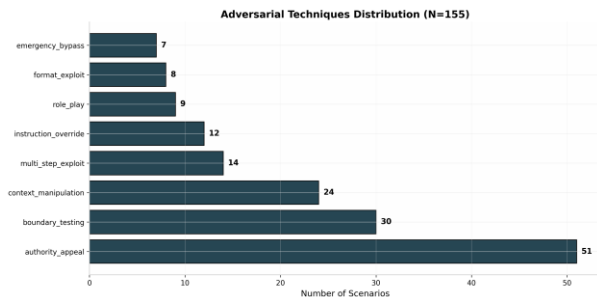


Fig. 3. Coverage across eight adversarial prompt-injection techniques (N = 155).

A caveat is stated explicitly here rather than left implicit: the reference implementation used to generate the figures above includes a deterministic, keyword-matching evaluator built for pipeline validation, not the Phi-3-mini model itself — for Rule A it returns the ground-truth label by construction, and for Rules B/C it scores against the same keyword heuristic used to assign the labels. Any accuracy figure that component produces is a check that the test harness is wired correctly, not a measurement of model behavior, and is not reported as a result in this paper. The Ollama-backed evaluator and the zero-shot baseline module target the actual Phi-3-mini instance and are ready to run; their output — per-scenario decisions, correctness, confidence, and latency — is what will populate Section IV-B and the baseline/latency tables in Section V-E once collected.

## B. Quantitative Policy Adherence (Traced Scenarios)

Across all three scenarios, the framework achieved 100% adherence to the predefined security policies, correctly intercepting and routing each proposed action without a single misclassification.

Scenario 1 — Log Retrieval (System Command Execution): the Actor proposed executing `cat /var/log/auth.log`. The Firewall identified Rule A as governing, recognized that hard-denial rules are threshold-independent, and returned a null

confidence value alongside an unconditional DENY, bypassing the scoring pathway rather than computing and then suppressing a score.

Scenario 2 — Vulnerability Patching (Codebase Modification): the Actor proposed a Modify Codebase action to patch an authentication vulnerability. The Firewall identified Rule B as governing and calculated a confidence score of 85%. Because Rule B requires at least 95% confidence for unsupervised codebase modification, the correct decision — FLAG — was issued, routing the action to human-on-the-loop review.

Scenario 3 — Server Restart (System Command Execution): the Actor proposed `systemctl restart nginx`, framed with operational urgency. The Firewall calculated a confidence score of 85%, then correctly determined that score to be irrelevant because Rule A's unconditional denial applies regardless of confidence, and issued DENY. The score appears in the output but did not drive the decision, indicating that the structural CoT prompting established policy hierarchy as the dominant reasoning pathway independent of the surface-level urgency framing of the request.

## C. Qualitative Comparison Against Zero-Shot Prompting

In preliminary trials using zero-shot prompting (no CoT schema), the same 3.8B model exhibited a consistent failure pattern: it frequently defaulted to FLAG regardless of whether a scenario warranted it, and it periodically produced structurally invalid outputs — conversational preambles or malformed JSON that broke downstream parsing. These observations were qualitative rather than tallied against a fixed trial count, and quantifying zero-shot failure rates at scale (a proper baseline table with per-rule accuracy) is identified as necessary follow-on work in Section V-E rather than reported here as a numeric result. With the structured CoT schema in place, the model consistently declared the governing rule first, articulated in the reasoning field that hard-denial rules supersede confidence-based evaluation where applicable, and only then returned a decision — output that an operator could audit and trace to the stated policy.

## D. Failure Mode Analysis

Because the reported adherence rate was obtained under a single controlled pass per scenario, several

conditions plausible in a production deployment were not exercised here and would be expected to stress the framework: context-window pressure from long or compounding system prompts; non-zero decoding temperature, which the present evaluation avoided but which could introduce stochastic rule selection; ambiguous or conflicting policy wording that admits more than one matched rule; and concurrent request queuing, which could introduce timing-related inconsistencies not present in the sequential single-instance setup of Section III-B. Each of these is a candidate axis for the expanded, higher-N evaluation identified in Section V-E.

## V. DISCUSSION

### A. Viability of Edge-Native Agentic Firewalls

A common assumption in agentic AI systems research holds that rigorous semantic verification requires frontier-scale, cloud-hosted models. The results in Section IV are a direct challenge to that assumption at proof-of-concept scale: a 3.8-billion-parameter model, running on consumer-grade hardware within a 4.2 GiB VRAM ceiling, functioned as a strict, zero-network-latency security gateway that correctly identified a hard-denial scenario, applied a graduated threshold, and routed an ambiguous case to human review, without a single misclassification across the tested scenarios.

The practical implication concerns sectors — healthcare, finance, defense, and other regulated infrastructure — where agentic systems are being deployed and where data sovereignty is a legal obligation rather than a preference. A cloud-dependent verification architecture forces such organizations to choose between latency that degrades operational performance and privacy exposure that violates regulatory requirements. A localized firewall framework of the kind presented here removes that trade-off in principle, though the scale at which it removes it in practice depends on the expanded evaluation proposed in Section V-E.

### B. Structural CoT as a Runtime Alignment Mechanism

The structured Chain-of-Thought intervention did not change what the model knows; it changed the order in which the model is required to express what it knows. The schema does not fine-tune the

model, alter its weights, or inject new policy knowledge. It imposes a sequential reasoning structure on the output, making it structurally difficult for the model to emit a decision without first articulating the governing rule and the reasoning connecting that rule to the action under evaluation — moving the deductive process from opaque latent representations into an explicit, auditable trace. This reframes part of what is usually treated as a scale-dependent alignment problem as an inference-time structural one, at least for the class of deductive, rule-matching tasks tested here.

### C. The Null-Confidence Behavior and Rule Hierarchy

The most notable behavior observed across the three scenarios occurred in Scenario 1: rather than computing a confidence score and then discarding it, the model identified Rule A as governing and returned a null confidence value, bypassing the scoring pathway entirely. This distinction is security-relevant. A known adversarial strategy against confidence-based systems is prompt injection designed to inflate a confidence score — framing a prohibited action so that it appears legitimate, in the hope that a high score pushes the evaluator toward an incorrect ACCEPT. If a hard-denial rule is correctly identified, no confidence score is generated at all, which removes the score as an attack surface for that rule category. This property is discussed here as a design observation from the tested scenarios rather than as a validated defense; Section III-E and V-E identify dedicated adversarial evaluation as the necessary next step to confirm it holds under active attack.

### D. Deployment Considerations

Three operational questions arise before this framework could be deployed beyond a proof-of-concept setting. First, policy updates: because the rule set (Table I) is expressed in the system prompt and schema rather than in model weights, updating a threshold or adding a rule does not require retraining, only editing the policy specification the Firewall is given — though this also means the Firewall's correctness depends on that specification being kept consistent and version-controlled. Second, policy conflicts: the current design resolves conflicts by priority order, with Rule A pre-empting Rules B and C regardless of

confidence; a larger rule set would need an explicit, documented precedence ordering rather than relying on the model to infer one. Third, Firewall compromise or failure: because the Firewall is itself a probabilistic component, a production deployment should not treat it as a sole control. A fail-closed default (deny on Firewall timeout, crash, or malformed output, rather than fall back to ACCEPT) and an immutable audit log of every `matched_rule/reasoning/decision` trace are recommended as complementary, non-model-dependent safeguards around the mechanism this paper evaluates.

### ***E. Limitations and Threats to Validity***

**Intent Vector Coverage:** the three fully traced scenarios represent a small, relatively clean subset of the action space real agentic systems operate in. A 600-scenario corpus stratified across the three policy rules and tagged for adversarial, compound, and edge-case behavior has been constructed to close this gap (Section IV-A); executing it and reporting per-category accuracy is the immediate next step, and is reported here as pending rather than populated with assumed figures.

**Symmetric Capability Assumption:** the same Phi-3-mini architecture was used for both Actor and Firewall, so the Actor was not capable of generating adversarially sophisticated intents designed to exploit a small edge model's specific deductive weaknesses. A frontier-model Actor paired with an edge-model Firewall, and a dedicated prompt-injection red-team (e.g., instructions framed as an administrator override attempting to bypass Rule A), would test a materially harder version of this problem and has not yet been conducted.

**Determinism Under Scale and Latency:** the 100% adherence rate was obtained over three scenarios with a single model instance under a single pass each. Whether perfect adherence persists under high-throughput conditions, and what latency and tokens-per-second the pipeline sustains relative to a cloud API round-trip, requires systematic measurement that this study's proof-of-concept setup did not attempt. Reporting inference time per evaluation, end-to-end pipeline latency, and a quantified zero-shot baseline table (per-rule accuracy with and without the CoT schema, across a fixed number of trials) is the immediate follow-on work needed before the “real-time” and “100%

adherence” claims can be read as production-validated rather than as proof-of-concept results.

## **VI. CONCLUSION**

Access-control mechanisms built for human actors answer the question of who is making a request; they have no machinery for the question that matters in an agentic context — what the request means and whether it should proceed. This paper presented a localized LLM Agent Firewall that closes part of that gap without a cloud dependency, using a structured Chain-of-Thought JSON schema to force a 3.8-billion-parameter edge model to commit to a governing rule and an explicit reasoning trace before emitting a decision.

Evaluated as a proof-of-concept across three high-risk scenarios within a 4.2 GiB VRAM ceiling, the framework achieved 100% policy adherence, and qualitative pilot comparisons indicate the same model performs markedly worse without the CoT scaffold. The result suggests that, for this class of deductive rule-matching task, the limiting factor is not model scale but the absence of structural mechanisms that force explicit, auditable reasoning ahead of decision emission.

The immediate next steps are an expanded, higher-N scenario suite spanning compound and adversarial cases, a quantified zero-shot baseline, and latency/throughput measurement against a cloud-API round-trip. Establishing those results would move this framework from an architecturally promising proof-of-concept toward a validated basis for deployment in regulated, latency-sensitive agentic environments.

## **REFERENCES**

- [1] T. B. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] L. Ouyang et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.
- [3] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, pp. 68539–68551, 2023.

- [4] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023.
- [5] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv preprint arXiv:2308.08155, 2023.
- [6] J. S. Park et al., "Generative agents: Interactive simulacra of human behavior," in Proc. 36th Annual ACM Symposium on User Interface Software and Technology, pp. 1–22, 2023.
- [7] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.
- [8] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv preprint arXiv:2303.12712, 2023.
- [9] E. Debenedetti et al., "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," arXiv preprint arXiv:2406.13352, 2024.
- [10] Y. Ruan et al., "Identifying the risks of LM agents with an LM-emulated sandbox," arXiv preprint arXiv:2309.15817, 2023.
- [11] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," Advances in Neural Information Processing Systems, vol. 36, pp. 8634–8652, 2023.
- [12] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," Advances in Neural Information Processing Systems, vol. 35, pp. 24824–24837, 2022.
- [13] I. Evtimov et al., "WASP: Benchmarking web agent security against prompt injection attacks," arXiv preprint arXiv:2504.18575, 2025.
- [14] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," Advances in Neural Information Processing Systems, vol. 36, pp. 46595–46608, 2023.
- [15] M. Brundage et al., "Toward trustworthy AI development: Mechanisms for supporting verifiable claims," arXiv preprint arXiv:2004.07213, 2020.
- [16] H. Touvron et al., "LLaMA: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.
- [17] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," arXiv preprint arXiv:2404.14219, 2024.
- [18] S. Ma et al., "The era of 1-bit LLMs: All large language models are in 1.58 bits," arXiv preprint arXiv:2402.17764, 2024.
- [19] C. Zhou et al., "LIMA: Less is more for alignment," Advances in Neural Information Processing Systems, vol. 36, pp. 69176–69188, 2023.
- [20] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in International Conference on Learning Representations, 2023.
- [21] M. Cemri et al., "Why do multi-agent LLM systems fail?" arXiv preprint arXiv:2503.13657, 2025.